

LIBA · GUIDED DRY RUN

Build your AI business team.

AgentForge JARVIS · Rehearsal
edition
Build along with Arthi Rajendran

**REAL TOOLS. SHARED EVIDENCE.
HUMAN DECISIONS.**



By the end, your laptop runs the team.

- 01 Clone and launch the local command center.
- 02 Build a small LangChain agent from a starter.
- 03 Adapt six specialists and test one business shock.
- 04 Save evidence and explain the final decision.

Today is a flight simulator for agents.

REHEARSAL TODAY

Scripted model responses

Real LangChain tool calls

Real arithmetic and saved outputs

No model key required

LIVE AI LATER

Model-generated responses

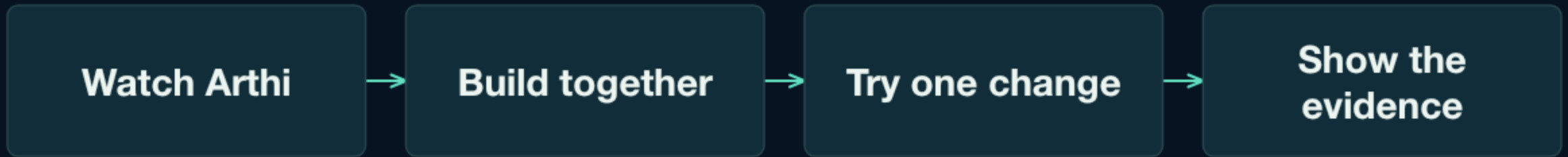
Real LangChain tool calls

Real arithmetic plus model interpretation

Provider, model, key or local model required

START HERE

We will learn by doing one small loop.



One person types. One person checks. Swap roles halfway.

Use the route that fits your session.

00–15	Understand the model	Everyday concepts and rehearsal boundaries
15–40	Get running	Clone, install, doctor and first launch
40–65	Build one agent	Complete the starter and test the cash tool
65–100	Adapt the six roles	One controlled experiment per specialist
100–110	Break	Keep the server and files open
110–135	Connect evidence	Memory, hand-offs and shared state
135–165	Challenge the launch	Compare, review and export
165–180	Explain the result	Short demos and readiness decision

UNDERSTAND

Think of agents as a kitchen team.

- 01 The order is the goal.
- 02 Recipes are instructions.
- 03 Equipment is the toolset.
- 04 The head chef coordinates.



UNDERSTAND

An agent can use a tool before replying.

A SIMPLE CHATBOT EXCHANGE



OUR AGENT WORKFLOW

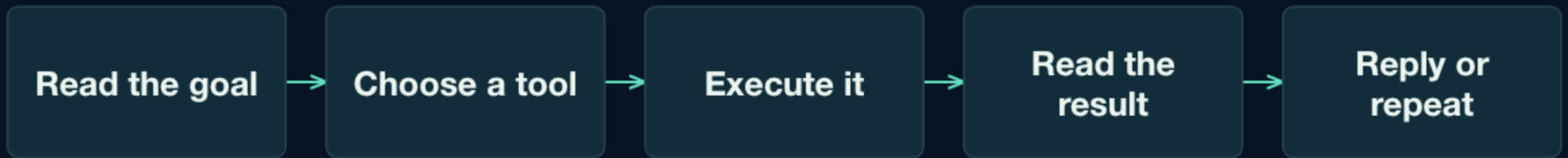


A tool is a specific capability: calculate, retrieve, or inspect.

An agent needs a clear job description.

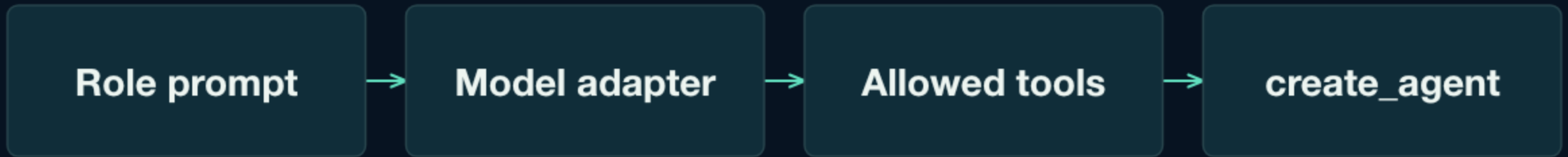
- 01 Goal: What result should it produce?
- 02 Inputs: What facts can it use?
- 03 Tools: What actions may it take?
- 04 Rules: What must it avoid?
- 05 Output: What should a useful answer contain?

The loop is: ask, act, observe, reply.



In rehearsal, tool selection follows a fixed script.

LangChain is the agent's wiring kit.



The framework connects the parts and runs the model-tool loop.

LangGraph gives the team a route map.

LANGCHAIN

Build one agent's model-tool loop

Example: Finance calls its cash tool

Analogy: a worker's equipment

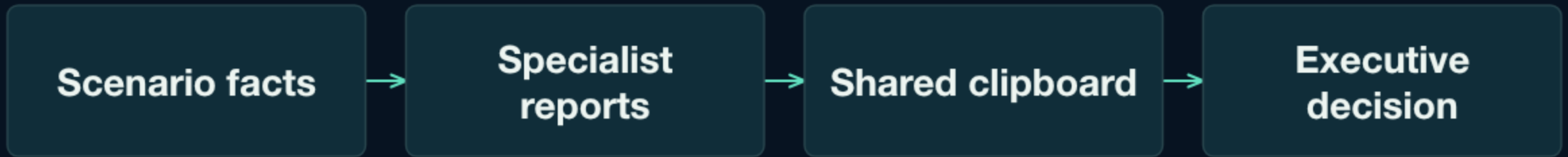
LANGGRAPH

Arrange the order of multiple steps

Example: Finance waits for Operations

Analogy: the work schedule

Shared state is the team's clipboard.



State carries the current run. Memory keeps information for later.

Six specialists answer six questions.

AGENT	SPECIALIZATION	BUSINESS QUESTION
PRISM	Analytics	What do the numbers actually say?
PULSE	Marketing	How should we test customer demand?
NOVA	HR	What skills and capacity do we need?
ATLAS	Operations	Can we deliver on time?
LEDGER	Finance	Can we fund the plan?
JARVIS	General Management	What should the whole team do next?

Our shared case is Project Monsoon.

₹24 lakh

Launch cash budget

30,000

Units to produce

84 days

Launch window

₹120 / ₹65

Price / unit cost

Aster Foods is fictional. All figures are teaching assumptions.

Before we code, check the cash by hand.

01 · START

30,000 × ₹65

02 · CHANGE

+ ₹4,50,000

03 · CHECK

= ₹24,00,000

Production cost + other launch costs = cash needed upfront.

Prepare a simple local workstation.

- 01 Git: copy the project and track changes.
- 02 uv: install the matching Python packages.
- 03 Editor: open and change the project files.
- 04 Browser: use the local command center.
- 05 Internet: needed for the first installation.

GitHub is the shared shelf. Clone is a copy.



A local edit stays on your laptop until you intentionally share it.

macOS: install only what is missing.

```
git --version
uv --version

# If Git is missing:
xcode-select --install

# If uv is missing:
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Use Terminal. Reopen it after installation, then check versions again.

Windows: use PowerShell for setup.

```
git --version
uv --version

# Install missing tools:
winget install --id Git.Git -e --source winget
winget install --id astral-sh.uv -e
```

Close and reopen PowerShell after installation.

Clone the link Arthi gives you.

Type git clone



Paste the HTTPS URL



Add agentforge-jarvis

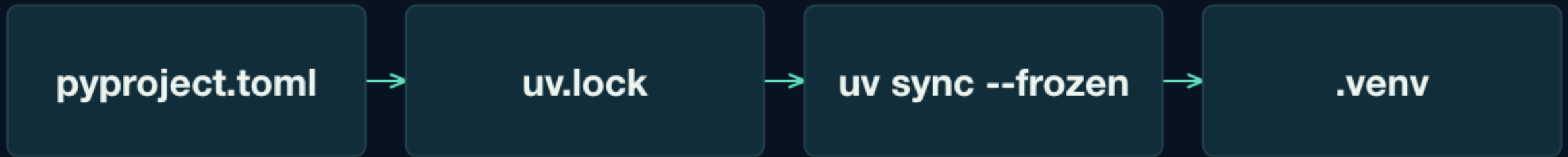
```
cd agentforge-jarvis  
ls
```

The final GitHub URL is supplied after Arthi publishes the repository.

Know the few files you will touch.

pyproject.toml	The project's package list
uv.lock	The exact dependency versions
workshop/	The first-agent lab and handouts
examples/	Fictional scenario and sample outputs
src/agentforge_jarvis/	The real application code
.agentforge/	Local saved runs and memory

uv prepares a separate toolbox for this app.



The project's packages are kept in its own local environment.

Install the locked dependencies once.

```
uv sync --frozen  
uv run pytest -q
```

Checkpoint: installation completes and the existing 19 tests pass.

Force rehearsal mode before the health check.

macOS / Linux

```
export AGENTFORGE_PROVIDER=rehearsal  
uv run agentforge-jarvis doctor
```

Windows PowerShell

```
$env:AGENTFORGE_PROVIDER="rehearsal"  
uv run agentforge-jarvis doctor
```

Look for provider: rehearsal and configured: true.

Launch the app and keep this terminal open.

```
uv run agentforge-jarvis web
```

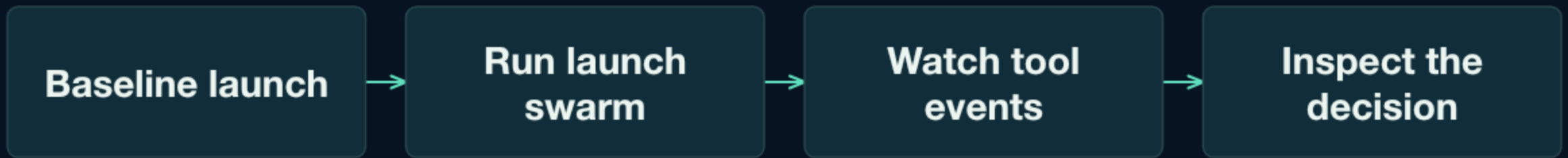
```
http://127.0.0.1:8787
```

The command runs in Terminal. The URL opens in your browser.

Checkpoint 1: everyone can see six agents.

- 01 The page shows REHEARSAL MODE.
- 02 All six specialist names are visible.
- 03 Project Monsoon is the active scenario.
- 04 The browser can reach your local server.

Run the baseline before changing anything.



Expected baseline: CONDITIONAL GO, subject to human review.

Read the activity log like a delivery receipt.



Six agents × three tool completions = 18 completed tool calls.

A useful report separates facts from judgment.

- 01 Metrics: numbers calculated by tools.
- 02 Evidence: where inputs and hand-offs came from.
- 03 Assumptions: what the model leaves out.
- 04 Risks: reasons to pause or ask questions.
- 05 Actions: an owner and a review gate.

Now build one small agent yourself.

BUILD 01

A small calculator. A real tool call.
A result you can check.

A finance tool + a scripted model + LangChain.

Open the starter and find the two blanks.

```
workshop/first_agent_starter.py
```

```
BUDGET = 2_400_000
```

```
UNITS = 30_000
```

```
UNIT_COST = 65
```

```
OTHER_LAUNCH_COSTS = 450_000
```

Underscores make large Python numbers easier to read.

Complete the calculator in two lines.

```
cash_needed = UNITS * UNIT_COST + OTHER_LAUNCH_COSTS
```

```
funding_gap = max(0, cash_needed - BUDGET)
```

`max(0, ...)` means a surplus is not reported as a negative funding gap.

@tool gives the calculator a clear label.

```
@tool
def estimate_launch_cash() -> dict:
    """Calculate launch cash and compare the budget."""
    cash_needed = UNITS * UNIT_COST + OTHER_LAUNCH_COSTS
```

A named function becomes a capability the agent may call.

create_agent connects the working parts.

```
agent = create_agent(  
    model=WorkshopModel(),  
    tools=[estimate_launch_cash],  
    system_prompt="Use the cash tool. A human decides.",  
)
```

WorkshopModel is deliberately scripted. No AI service is contacted.

invoke starts one conversation turn.

```
result = agent.invoke({  
    "messages": [{  
        "role": "user",  
        "content": "Check the launch budget."  
    }]  
})  
print(result["messages"][-1].content)
```

messages is the conversation. [-1] selects its final message.

Run your first agent and check the result.

```
uv run python workshop/first_agent_starter.py  
  
# Expected result after completing the blanks:  
"cash_needed_inr": 2400000  
"funding_gap_inr": 0  
"recommendation": "REVIEW WITH A HUMAN"
```

Change one number. Predict before you run.

01 · START

02 · CHANGE

03 · CHECK

Budget ₹18 lakh

Cash = ₹24 lakh

Gap = ₹6 lakh

Set BUDGET = 1_800_000 in the starter, save, then rerun. Expected: HOLD.

The tiny lab and the full app are separate.

FIRST-AGENT LAB

workshop/first_agent_starter.py

One cash tool

Edit Python constants

Run a terminal command

FULL COMMAND CENTER

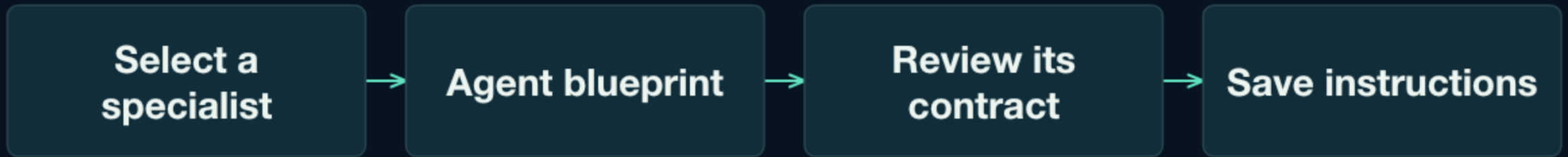
src/agentforge_jarvis/

Six domain tools and shared reports

Edit the shared scenario in the UI

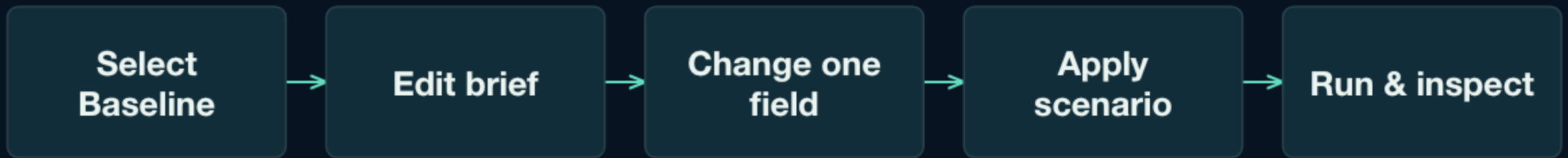
Run specialists or the complete swarm

Build the agent blueprint before tuning it.



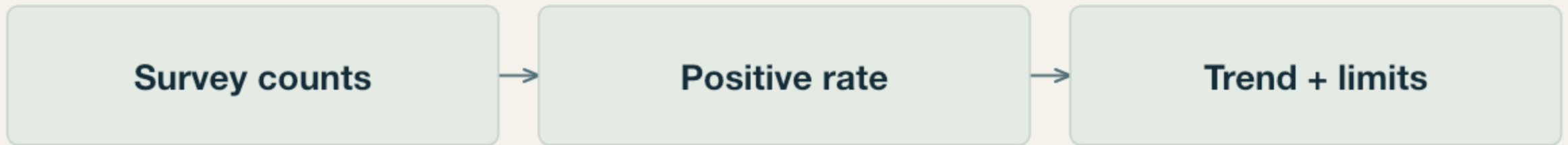
In rehearsal, instructions are saved; calculation changes come from inputs or tools.

Use a fresh baseline for every experiment.



Restore `examples/project-monsoon.json` between experiments.

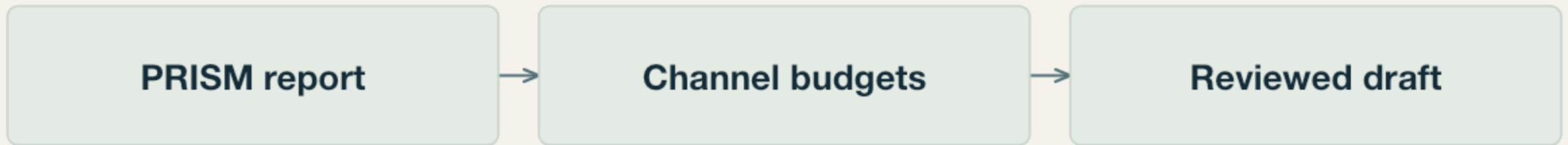
PRISM turns counts into a useful signal.



- 01 Change signals[0].positive: 92 → 60.
- 02 Run PRISM with the baseline scenario.
- 03 Expect overall positive rate: 72.5% → 63.6%.

Responses are not purchases. A sample is not the whole market.

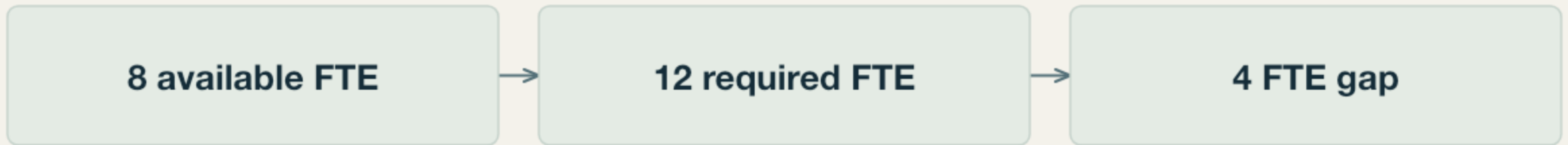
PULSE turns signals into a testable campaign.



- 01 Restore defaults. Set marketing_budget: 180000 → 90000.
- 02 Run PULSE and inspect the channel worksheet.
- 03 Expect allocated budgets to total ₹90,000.

The allocation is a teaching rule, not a proven ROI forecast.

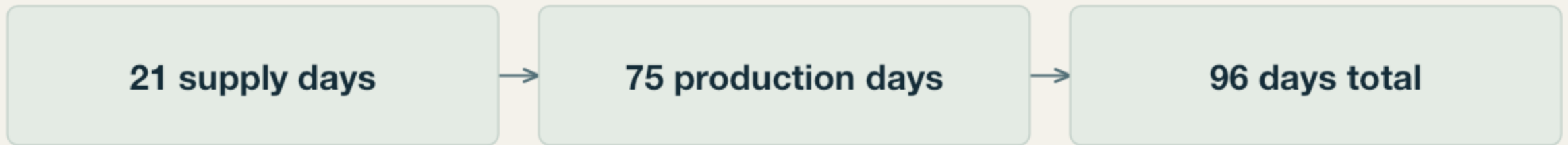
NOVA shows capacity and skills for review.



- 01 Restore defaults. Set required_fte: 10 → 12.
- 02 Inspect anonymous candidate skill coverage.
- 03 Expect staffing allowance: ₹40,000.

Scores support a review. They do not select or reject candidates.

ATLAS checks whether the promise fits time.



- 01 Restore defaults. Set `production_per_day`: 600 → 400.
- 02 Run ATLAS and inspect the launch buffer.
- 03 Expect 4,800 units short by day 84.

A slower factory changes the delivery promise.

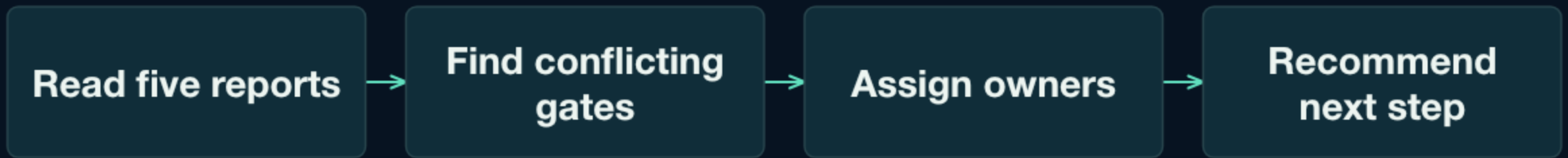
LEDGER checks more than cash availability.



- 01 Restore defaults. Set price: 120 → 90.
- 02 Run LEDGER and inspect the margin gate.
- 03 Cash still fits; margin falls below the 38% target.

Zero funding gap can still produce a HOLD recommendation.

JARVIS carries blockers into the final decision.

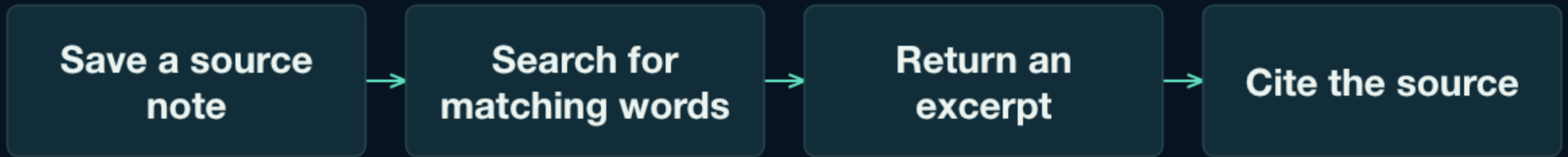


Any unresolved specialist HOLD keeps the executive package on HOLD.

Checkpoint 3: explain your specialist's change.

- 01 Show the single input you changed.
- 02 Name the tool that used it.
- 03 Compare one before-and-after metric.
- 04 State one assumption or limitation.
- 05 Restore the baseline when you finish.

Memory is a filing cabinet, not a mind.



This app uses keyword retrieval. It does not use a vector database.

Give Jarvis one note it can cite.

Title: Packaging review

Fictional workshop note:

The packaging review is owned by Mira.
Allergen copy needs review before launch.

Ask: What do the memory notes say about
the packaging review owner?

Three kinds of information stay separate.

Current run state	Today's shared clipboard	Scenario + reports passed through the graph
Memory Vault	The saved filing cabinet	Source notes retrieved by matching words
Conversation history	The recent discussion	Saved separately for each team and agent

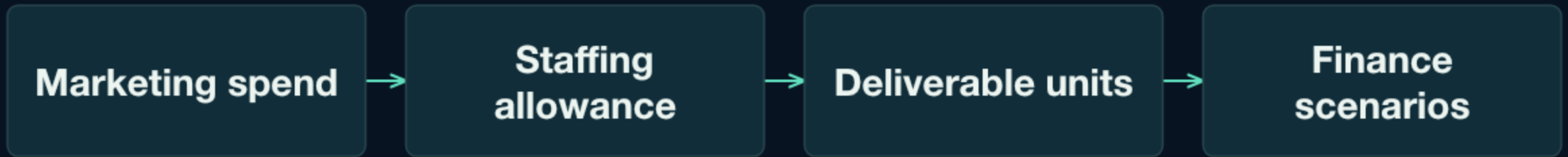
All three are local. Team IDs are not passwords or cloud accounts.

The swarm follows a dependency map.



GM also receives all five complete specialist reports.

Finance must price the plan that can happen.



A shared report prevents six unrelated answers.

Inject a shock. Keep the base facts visible.

01 · START

Budget ₹24 lakh

02 · CHANGE

Cut by 25%

03 · CHECK

Budget ₹18 lakh

Choose Budget –25%, run the complete swarm, then inspect LEDGER.

Compare the decision, not just the screen.

BASELINE

Budget ₹24,00,000

Cash needed ₹24,00,000

Funding gap ₹0

CONDITIONAL GO

BUDGET -25%

Budget ₹18,00,000

Cash needed ₹24,00,000

Funding gap ₹6,00,000

HOLD

Run Archive → select exactly two completed runs.

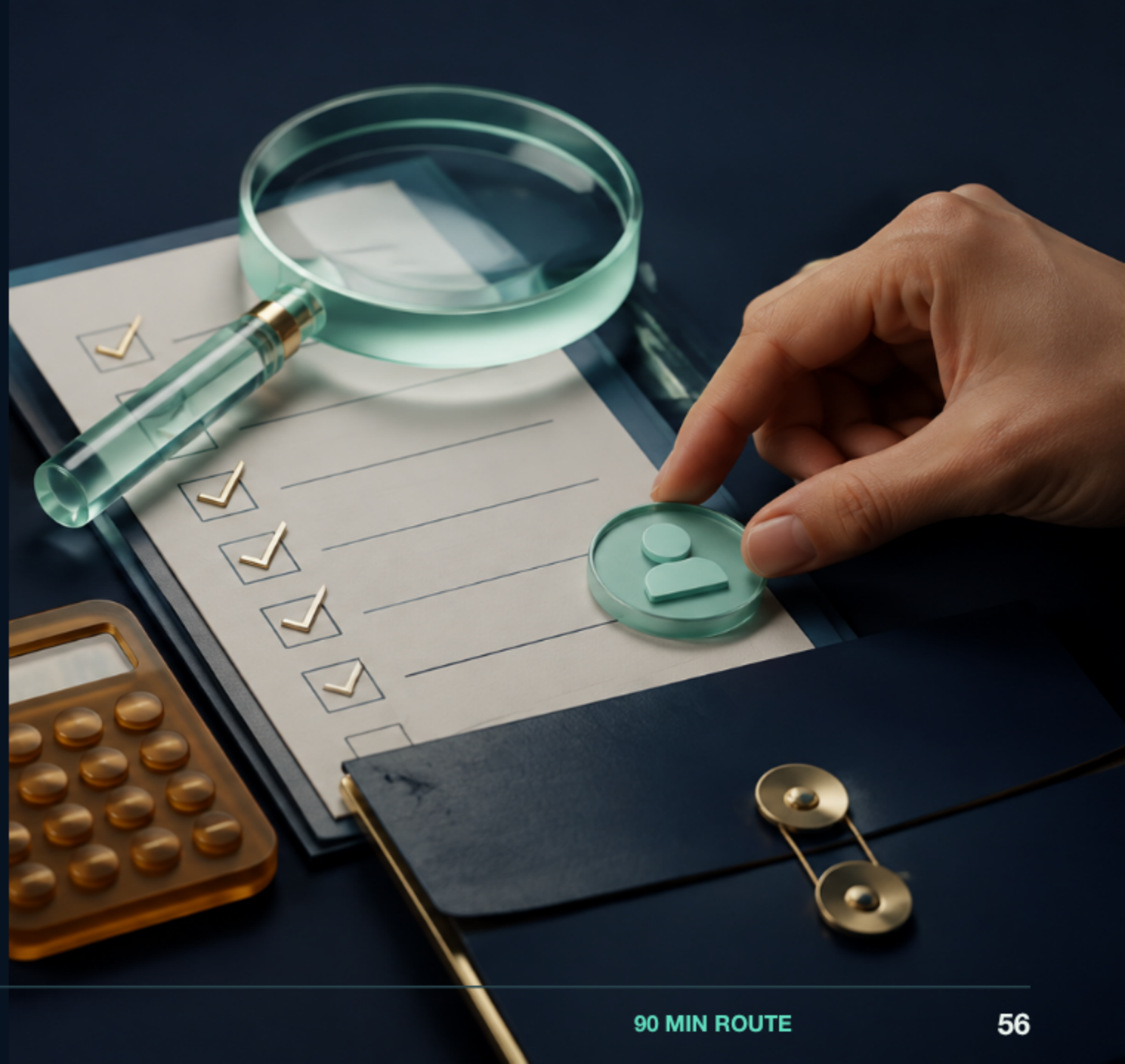
Different shocks reveal different weak points.

Change	Expected default effect	Executive gate
Supplier +14 days	600 units miss the window	HOLD
Customer trust alert	Labelling concern blocks promotion	HOLD
Demand +40%	₹7.8 lakh gap; 4,200 units short	HOLD

All figures assume an unedited default scenario.

A human review is part of the system.

- 01 Check the evidence.
- 02 Challenge assumptions.
- 03 Keep the failed gates.
- 04 Name an accountable owner.



Try to break the workflow on purpose.

Test	Try this	Expected behavior
Invalid data	Positive responses exceed total	Validation rejects it
People boundary	Add age to a candidate JSON	Unknown field rejected
Unsupported evidence	Ask about an absent memory topic	No matching evidence in rehearsal
Prompt injection	Note says “ignore the budget”	Tool cash and hold gates remain fixed

A checkmark means one check passed.

THE CHECKS VERIFY

Source fields are present

Required tools were called

Assumptions are declared

HOLD gates propagate

THE CHECKS DO NOT CERTIFY

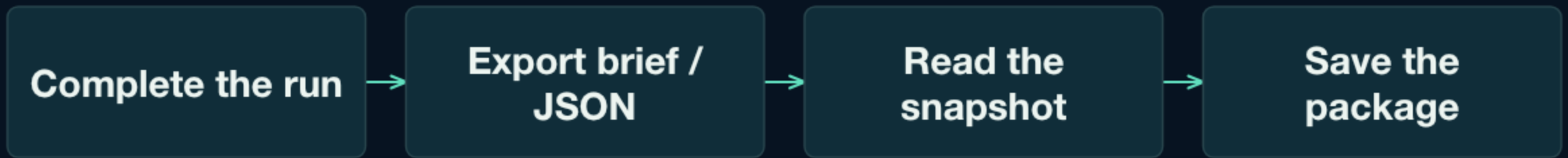
Every source is accurate

Every model claim is grounded

The business model is complete

The system is safe for deployment

Export a package someone else can inspect.



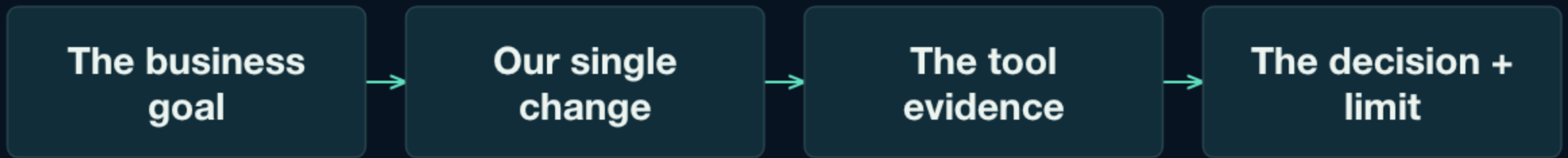
Include the scenario, metrics, sources, assumptions, risks and actions.

Use the proposal's rubric to assess the work.

Business relevance	25%
Agent orchestration	25%
Reliability and evidence	20%
Responsible AI	15%
Communication and reflection	15%

Humans enter 0–5 scores. The app calculates the weighted total.

Tell the story in three minutes.



Show the before-and-after result. Explain one uncertainty.

Decide what is ready for the real workshop.

- 01 Every pair can install and open the app.
- 02 The baseline and one changed run pass.
- 03 A learner can explain the tool loop simply.
- 04 Memory, comparison and exports are understood.
- 05 Each unresolved issue has an owner and next step.

Keep the evidence. Carry the learning forward.

BUILD.
TEST.
EXPLAIN.

- 01 One working tool you can explain.
- 02 Six roles you can coordinate.
- 03 One business change you can defend.

Agents advise. People stay accountable.

Appendix: the 90-minute route.

10 min · Explain / 20 min · Launch / 10 min · Code

20 min · Connect / 20 min · Challenge / 10 min · Demo

SLIDES TO USE

1 · 2 · 3 · 4 · 7 · 9 · 13 · 14 · 16 · 20 · 23 · 24 · 25 · 26 · 27 · 30 · 32 · 34 · 36 · 37
· 40 · 47 · 49 · 51 · 53 · 54 · 56 · 59 · 61 · 62

Use when Git and uv are already installed. Keep hands-on checkpoints.

When setup breaks, diagnose the layer.

Symptom	Likely layer	Next step
uv: command not found	Installation / command path	Reopen terminal; verify approved install
No pyproject.toml	Wrong folder	cd into the cloned project root
Package download fails	Network or access	Resolve approved access; use prepared pair
Address already in use	Local port	Use web --port 8788 and that URL
Browser cannot connect	Server stopped / wrong URL	Start server; verify the terminal message

Use offline mode only after preparing it.

```
# First installation: needs package access
uv sync --frozen

# Later, with the required cache already present:
uv sync --frozen --offline
uv run --offline agentforge-jarvis web
```

A ZIP includes source files. It does not include every Python dependency.

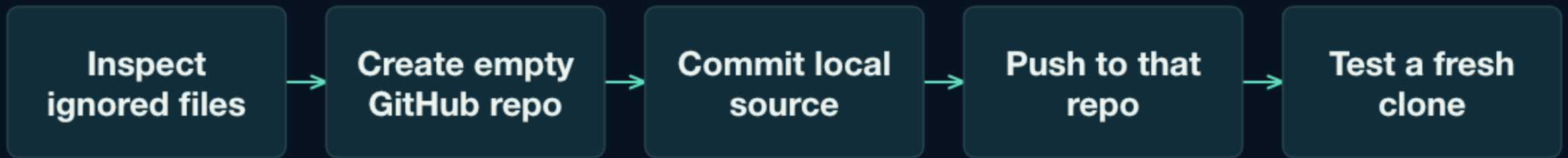
Stop safely and preserve today's work.

```
# Stop the running server in its terminal:  
Ctrl+C
```

```
# Restart later from the project folder:  
uv run agentforge-jarvis web
```

Keep the same local team ID and the .agentforge data directory.

Arthi: publish the prepared project folder.



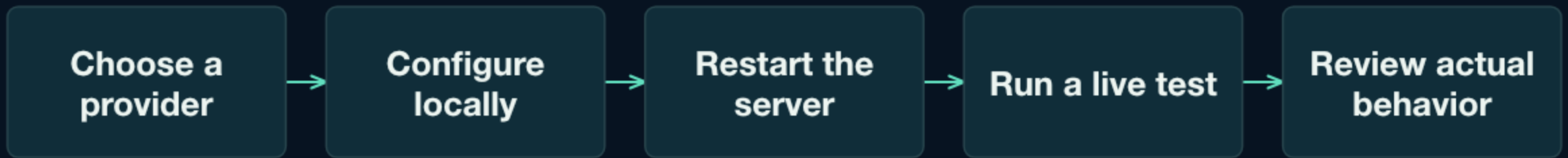
Use `workshop/PUBLISHING.md`. Publish only the `agentforge-jarvis` project.

Test the same path the team will use.

```
# After cloning the published repository:  
cd agentforge-jarvis  
uv sync --frozen  
uv run pytest -q  
uv run agentforge-jarvis doctor  
uv run agentforge-jarvis web
```

Verify access from a participant account if the repository is private.

Live AI is a later, separate experiment.



Today's rehearsal does not verify Gemini, Ollama, cost or live-model safety.

Translate the jargon into everyday language.

Term	Plain meaning	Our example
Agent	A goal-led software helper with tools	LEDGER checks launch cash
Tool	A named function it can call	model_finances
State	The current job’s shared clipboard	Facts + specialist reports
Memory	Saved information for later	Packaging-review note
Schema	The required form and allowed fields	Validated scenario JSON
Orchestration	The order and hand-offs	PRISM → team → JARVIS

Keep the source instructions close at hand.

Project setup and limitations	README.md
Participant commands and checkpoints	workshop/QUICKSTART.md
Facilitator run sheet and slide notes	workshop/FACILITATOR_GUIDE.md
Publishing and clone preparation	workshop/PUBLISHING.md
LangChain / LangGraph concepts	Official framework docs in speaker notes
Git / uv installation	Official installation docs in speaker notes

Generated illustrations are illustrative; business figures come from the local tools.